CO1 – ASSIGNMENT 3

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

Student: Sudharsan R **Register No.:** 1912260284

Course: C Programming **Submission date:** 27-10-2026

TASK 1 – Implement and Test the Billing Program (4 Marks)

Complete C program

```
#include <stdio.h>
#include <limits.h>

#define MODULUS 65536U
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE1_PAISE 0LL
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
#define FIXED_CHARGE_PAISE 5000LL

typedef unsigned short meter_t;

int meter_consumption(unsigned int previous, unsigned int current,
                      unsigned int *units)
{
    if (previous > 65535U || current > 65535U || units == NULL)
        return 0;
    if (current >= previous)
        *units = current - previous;
    else
        *units = (MODULUS - previous) + current; /* one rollover */
    return 1;
}

long long compute_bill(unsigned int units)
{
    long long bill = FIXED_CHARGE_PAISE;
    if (units <= SLAB1_UNITS)
        bill += (long long)units * RATE1_PAISE;
    else if (units <= SLAB2_UNITS)
        bill += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    else {
        bill += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
        bill += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    }
    return bill;
}

void print_bill(unsigned int units)
{
    long long p = compute_bill(units);
    printf("%u units -> Rs %lld.%02lld\n", units, p / 100, p % 100);
}

int main(void)
{
    unsigned int tests[] = {0, 99, 100, 101, 150, 199, 200, 201, 350};
    unsigned int n = sizeof tests / sizeof tests[0];

    printf("Meter type size: %zu bytes; USHRT_MAX=%u\n",
           sizeof(meter_t), (unsigned int)USHRT_MAX);
    for (unsigned int i = 0; i < n; ++i)
        print_bill(tests[i]);

    unsigned int units;
    if (meter_consumption(65400U, 120U, &units))
        printf("Rollover consumption: %u units\n", units);

    if (!meter_consumption(70000U, 120U, &units))
        printf("Invalid meter reading rejected\n");

    return 0;
}
```

Expected output on a system where unsigned short is 2 bytes and maximum 65535:

```
Meter type size: 2 bytes; USHRT_MAX=65535
0 units -> Rs 50.00
99 units -> Rs 50.00
100 units -> Rs 50.00
101 units -> Rs 52.25
150 units -> Rs 162.50
199 units -> Rs 272.75
200 units -> Rs 275.00
201 units -> Rs 279.50
350 units -> Rs 950.00
Rollover consumption: 256 units
Invalid meter reading rejected
```

Test interpretation

The boundary tests show the first 100 units are free, units 101–200 cost ■2.25 each, and units above 200 cost ■4.50 each. A 16-bit meter reading that falls from 65400 to 120 is treated as one rollover: (65536 − 65400) + 120 = 256 units. Readings outside 0–65535 are rejected. In a real application, a timestamp or rollover counter should be used if more than one rollover could occur between readings.

Array/loop alternative and complexity

Tariff limits and rates could be stored in arrays and processed by a loop. If there are s tariff slabs, the loop takes O(s) time and O(s) space for the tariff arrays. The current implementation uses a fixed number of conditions, so billing is O(1) time and O(1) extra space. A loop is easier to extend when tariffs become numerous or configurable; fixed branches are simple for only three slabs.

TASK 2 – Compare Expectations with Execution (4 Marks)

1. Float versus integer-paise totals

The integer-paise version stores every charge as an exact integer. Floating-point values such as ■2.25 are commonly represented exactly or closely on binary systems depending on the value, but decimal fractions such as ■0.10 are not generally exact. A floating-point total can therefore differ slightly from an integer-paise total. For the given tariff, the per-unit rates are whole paise, so integer paise is the most reliable method.

2. Sample total for scale comparison

Households/bills	Example average bill	Total	Fits signed 32-bit int paise?
4,000	■500	■20,00,000 = 200,000,000 paise	Yes
60,000	■500	■3,00,00,000 = 3,000,000,000 paise	No (exceeds 2,147,483,647)
800,000	■500	■40,00,00,000 = 40,000,000,000 paise	No

These are illustrative totals assuming ■500 average bill, not measured bills. They show why a 32-bit signed integer is insufficient for district-wide totals in paise. A 64-bit signed integer is recommended. An int total in rupees might fit these examples but cannot represent paise exactly; use a wide integer paise total.

3. Type sizes and limits

C only guarantees minimum ranges for standard integer types; exact sizes vary by implementation. Use sizeof and limits.h on the target compiler. Common systems use 4-byte int and 8-byte long long, but the report must use the actual measurements from the lab system. A 64-bit signed integer has a maximum of 9,223,372,036,854,775,807, comfortably above the illustrative 40 billion paise total. Floating-point overflow is less likely at this scale, but precision loss can occur before overflow.

Alternative: double district total

A double accumulator typically uses 8 bytes and has a large range, but it cannot exactly represent every decimal fraction. At 800,000 bills, small per-bill errors may accumulate; the exact error depends on input values and summation order. A long long accumulator also typically uses 8 bytes, is exact for integer paise, and costs O(1) time per addition. Both approaches use O(1) space per accumulator; integer paise is preferred for correctness.

TASK 3 – Recommend and Defend the Final Design (4 Marks)

Recommended design

• Meter readings: use a verified 16-bit unsigned type (uint16_t where available) for the meter register and a wider unsigned integer for consumption calculations.

• Money: store tariff rates and bills as integer paise; use long long or int64_t for totals.

• Tariffs: centralise fixed charge, slab limits and rates in one header or configuration module so a revision is made once.

• Validation: reject invalid readings, negative units, malformed input and impossible register values; handle rollover explicitly.

• Testing: run zero, slab boundaries, rollover, invalid-input and hand-calculated bill cases; retain test evidence.

• Debugging: compile debug output conditionally with #ifdef DEBUG, and compile normal builds without it.

Limitations and tariff changes

The basic model assumes one rollover at most between readings and a single tariff for all consumers. Smart meters sending readings every 15 minutes make multiple rollovers between readings unlikely, but timestamps and meter IDs should still be checked. A time-of-day tariff requires separate consumption totals by time band; simply applying one rate to all units would be wrong. If the tariff changes, update the central configuration or load a validated configuration file.

Configuration file alternative

Reading tariff values from a configuration file lets authorised staff change rates without recompiling. It adds start-up parsing time O(s) for s tariff entries and O(s) storage for those entries. Recompiling constants has no start-up parsing cost but requires rebuilding and redistributing the program after every tariff revision. For a co-operative that expects frequent tariff changes, a validated configuration file is preferable, with versioning, audit logs and error handling.

DAY 4 DESIGN CHALLENGE – TASK 4: Re-evaluate and Adapt the Design (8 Marks)

Changed requirements

The new smart meters report watt-hours every 15 minutes, approximately 35,040 readings per household per year. Units consumed must be computed from cumulative energy readings or interval readings, and a time-of-day tariff adds a 20% surcharge for energy used from 6 PM to 10 PM. Tariff values must be changeable without recompiling.

Decision: MODIFY the original design

Keep integer-paise accounting, explicit validation and modular functions, but modify the data model and tariff source. Store meter energy readings in a sufficiently wide integer type, such as uint64_t if the meter specification supports it. If values are cumulative, compute the difference between readings and handle meter rollover using the meter's documented modulus. If readings are interval watt-hours, validate each interval and aggregate into the correct time band. Store tariff configuration in a file read at start-up, and validate all limits and rates before processing bills.

Time-of-day calculation

For each interval, determine the time band from its timestamp. For intervals fully within 6 PM–10 PM, apply the 20% surcharge; for intervals outside it, use the base tariff. If an interval crosses 6 PM or 10 PM, split the interval at the boundary or obtain finer-grained meter data so energy is allocated correctly. Do not infer the whole interval's tariff from only its end time.

Suggested data model

```
typedef struct {
    unsigned long long energy_wh;
    unsigned int tariff_category;
    /* timestamp and meter ID should also be stored */
} meter_reading_t;

/* Store currency as integer paise; use a wide integer total.
   Load base rates and slab limits from a validated config file.
   Use integer arithmetic and a documented rounding rule for any
   20% surcharge that produces a fractional paise. */
```

For a 20% surcharge, calculate in a wider integer numerator and divide only once with a documented rounding rule, for example (base_paise * 120 + 50) / 100 for non-negative values using round-half-up. Ensure the multiplication cannot overflow; validate the rate and use a sufficiently wide intermediate type.

Complexity, reliability and limitations

For n readings and s tariff slabs, a direct fixed-slab calculation is O(n) overall; a loop over tariff slabs is O(n·s). With only a few slabs, both are efficient. Configuration parsing takes O(s) time and O(s) space once at program start. A single reading can be processed in O(1) space, while storing a year's full readings requires O(r) space for r readings per household. The design must also handle missing/out-of-order readings, clock changes, daylight-saving rules where relevant, duplicate records, meter resets, configuration errors and audit requirements.

Conclusion

The recommended solution retains integer-paise billing and explicit validation, while modifying the design for higher-resolution smart-meter data and configurable tariffs. Wide integer types support district-scale totals, and timestamp-aware allocation ensures the evening surcharge is applied only to eligible energy. Test results should be recorded from the actual compiler and system before final submission.